A Learned Cuckoo Filter for Approximate Membership Queries over Variable-sized Sliding Windows on Data Streams

YAO TIAN, The Hong Kong University of Science and Technology, China
TINGYUN YAN, Guangzhou University, China
RUIYUAN ZHANG, The Hong Kong University of Science and Technology, China
KAI HUANG, The Hong Kong University of Science and Technology, China
BOLONG ZHENG, Huazhong University of Science and Technology, China
XIAOFANG ZHOU, The Hong Kong University of Science and Technology, China

Designing a space-efficient data structure to answer membership queries while ensuring high accuracy and real-time response is a challenging task in the field of stream processing. Many techniques have been developed to answer these queries in a sliding windows manner. However, assuming the user will conduct the query with the presupposed window size is not always practical. In this paper, we introduce a novel data structure called Learned Cuckoo Filter (LCF). It can provide satisfactory results for the approximate membership query on data streams, regardless of the user-defined query windows. LCF operates by adaptively maintaining cuckoo filters with the assistance of a well-trained oracle that learned the frequency feature of the data within the stream. To further enhance memory utilization, we develop a compact version of LCF (denoted by LCF_C), which selectively removes redundant information to reduce space consumption without compromising query accuracy. Furthermore, we conduct a thorough theoretical analysis of query accuracy and provide detailed guidelines for optimal parameter selection (denoted by LCF_O). Extensive experimental studies on synthetic and real-world datasets demonstrate the superiority of the proposed methods in terms of both space consumption and accuracy. Compared to the state-of-the-art algorithms, LCF_O can reduce up to 61% of space cost at the same error level, and achieve up to 12× improved accuracy with the same space cost.

CCS Concepts: • **Information systems → Data structures**; **Query optimization**; **Data streams**; • **Theory of computation → Approximation algorithms analysis**; **Bloom filters and hashing**; • **Computing methodologies → Machine learning approaches**.

Additional Key Words and Phrases: Approximate Membership Query; Variable-sized Sliding Windows; Data Stream; Cuckoo Filter; Machine Learning

ACM Reference Format:
Yao Tian, Tingyun Yan, Ruiyuan Zhang, Kai Huang, Bolong Zheng, and Xiaofang Zhou. 2023. A Learned Cuckoo Filter for Approximate Membership Queries over Variable-sized Sliding Windows on Data Streams. *Proc. ACM Manag. Data* 1, 4 (SIGMOD), Article 264 (December 2023), 26 pages. https://doi.org/10.1145/3626758

Authors' addresses: Yao Tian, The Hong Kong University of Science and Technology, Hong Kong SAR, China, ytianbc@cse.ust.hk; Tingyun Yan, Guangzhou University, Guang Zhou, Guang Dong, China, tingyun_yan@e.gzhu.edu.cn; Ruiyuan Zhang, The Hong Kong University of Science and Technology, Hong Kong SAR, China, zry@ust.hk; Kai Huang, The Hong Kong University of Science and Technology, Hong Kong SAR, China, ustkhuang@ust.hk; Bolong Zheng, Huazhong University of Science and Technology, Wu Han, Hu Bei, China, zblchris@gmail.com; Xiaofang Zhou, The Hong Kong University of Science and Technology, Hong Kong SAR, China, zxf@cse.ust.hk.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.
2836-6573/2023/12-ART264 $15.00
https://doi.org/10.1145/3626758

1 INTRODUCTION

In many practical fields, such as e-commerce [24, 30, 31], networking [15], and big data systems [7, 36], the ability to detect recent occurrences of elements in a data stream is of great importance. Approximate membership query (AMQ) methodologies have been proven instrumental in this task, facilitating the rapid identification of an element's presence within a data stream while accepting a certain level of inaccuracies. Over the past few decades, various probabilistic data structures, *also known as* filters in the context of AMQs, have been developed, such as Bloom filters (BF) [6], cuckoo filters (CF) [18], and quotient filters (QF) [5]. The fundamental principle underlying these approaches involves the creation and maintenance of a compact data structure, which utilizes a particular strategy to record element occurrences. These structures demonstrate satisfactory performance in terms of accuracy, time, and space costs. A recent study [25] deployed machine learning models to further ease the space requirement by undertaking part of the queries using a pre-trained classifier. Meanwhile, some other AMQ solutions on infinite streaming data have been attempted by selectively recording the appearance of passed elements, such as [8, 15, 16, 19, 26, 27, 34, 36, 38–40, 44, 47]. It should be noted that these filters are traditionally designed for static or dynamic datasets with a fixed, pre-determined time frame known as the sliding window [13]. Under this concept, the window that maintains the most recent W_0 data elements serves as the basis for AMQ responses.

Nonetheless, in real-world applications, it is not practical to assume that the users will always query with W_0-size windows. It is more common to encounter requests for different query window sizes and levels of accuracy, as they reflect the users' varying needs. For example, the recommendation system aims to present users with ads for products in which they are interested, but the frequency for this can differ from user to user and product to product. Frequent duplicated ads can frustrate users. This necessitates altering the window sizes to filter out certain duplicates. Another example is network traffic analysis. Consider a web server log. The administrators can query the system if an IP address has visited the server in the last 5 minutes. If more investigation is needed, they can further check if the same IP address accessed the server in the past 1 hour. Such queries are common and helpful for analyzing user profiles, interests, and real-time network traffic.

As a result, it is important to develop a new data structure that can adapt to AMQs on a stream with flexible query windows and accuracy levels, determined by both the user and the application. Achieving zero false negatives (FNs) and minimal false positives (FPs) similar to Bloom filters across any query window size would be the ideal scenario. Yet, this is practically unattainable in streaming environments due to memory limitations. Allowing for minimal FNs when querying on large windows could be a necessary and acceptable compromise since old data is usually less important than recent ones. Recall the above example of the recommendation system: a few infrequent duplicated ads (representing a few FNs over a large window) have minimal impact on the user's experience but can still help to increase the ad's reach and click-through rate. [15] offers more similar use cases, such as detecting click inflation in the pay-per-click scheme. Therefore, this new structure should possess the following desired characteristics: 1) zero FNs and bounded false positive rate (FPR) when the query window is shorter or equal to the pre-defined window size W_0; 2) low FPR and false negative rate (FNR) when the query window is larger than W_0; 3) customizable accuracy levels of the various queries for the users, *e.g.,* provide higher accuracy on smaller window sizes; and 4) smaller space consumption and less response time than the state-of-the-art sliding window solutions at the same error level. However, this is a non-trivial task. Firstly, it is difficult to determine which elements should be kept or removed within the constraints of limited space in a one-pass stream manner. Secondly, a robust theoretical guarantee on query accuracy for arbitrary

window sizes is challenging. Thirdly, how to determine the optimal settings for the designed data structure that can meet the specific requirement of the users is a critical question as well.

Motivated by the above, in this paper, we propose Learned Cuckoo Filter (LCF), a learning-enhanced compact data structure to answer AMQs over variable-sized sliding windows. The idea behind our approach is the delicate fusion of two techniques: 1) conventional filters with formal guarantees and 2) a machine learning based-model that captures the features of the time interval between two consecutive occurrences of an item. To overcome the challenges posed by the variable window sizes, we propose the basic version of LCF, where cuckoo filters and a pre-trained oracle work together to make the decision of the presence of an item. The CFs can provide a solid accuracy guarantee for shorter windows, and the learned oracle can provide guidance for AMQs for arbitrary windows. Moreover, we develop a compact and optimized version of the base model. Inspired by the fact that the newest occurrence of an item in the filter determines the membership query results, we utilize the trained oracle to quickly nail down the historical position of each newly arrived element and remove this redundancy safely. By taking advantage of the partial-key cuckoo hashing, we rearrange the fingerprints and compact sparse CFs, which enables a lower level of space usage without hurting query performance. Lastly, we demonstrate how to deliver satisfactory answers under various windows and accuracy levels through comprehensive theoretical analysis, parameter optimizations, and extensive experiments.

To the best of our knowledge, this is the first work to develop a space-efficient, learning-enhanced filter with performance guarantees over variable-sized sliding windows of one-pass data streams. The main contributions of this paper include:

- We extend the AMQs over sliding windows on data streams to a more generalized and challenging setting where the window size is variable. To support such queries, we devise a novel learning-enhanced method called learned cuckoo filter (LCF).
- We develop two space optimization strategies to improve the space efficiency of LCF (denoted by LCF_C) that selectively erase redundant information without hurting query accuracy.
- We conduct a rigorous theoretical analysis of the upper bounds of expected FPR and FNR for different window sizes, based on which detailed guidelines for parameter settings are proposed to achieve optimal performance. The optimized LCF (denoted by LCF_O) achieves higher accuracy with the same space cost.
- Extensive experiments on real-world and synthetic data demonstrate that LCF can achieve up to 9× improved FPR and 134× improved FNR with the same space cost. LCF can also reduce space by an average of 50% over state-of-the-art sliding window algorithms at the same error level.

The rest of the paper is organized as follows. The related work is reviewed in Section 2. Section 3 introduces preliminaries and the research problem formally. The basic LCF and compact LCF are detailed in Section 4 and Section 5, with a theoretical analysis and parameter optimizations in Section 6. Section 7 presents the experimental results. We conclude this paper in Section 8.

2 RELATED WORK

In this section, we first review the cuckoo filter, which serves as a building block of our method, and then four types of filters closely related to our work.

Cuckoo filter. The cuckoo filter [18] is an efficient alternative to Bloom filters, with improved FPR and the ability to delete elements. The CF operates by using a cuckoo hash table with two hash functions. The hash table is an array of m buckets, with each bucket containing b entries. To save space, the CF stores only the fingerprint ξ_e, a fixed-length bit array derived from the element e, rather than the element itself. The two hash functions, h_1 and h_2, are used to locate possible

buckets when inserting, deleting, or searching for elements. These functions are defined as follows:

$$h_1(e) = hash(e), \tag{1}$$

$$h_2(e) = h_1(e) \oplus hash(\xi_e), \tag{2}$$

where $\oplus$ is the XOR operation. Note that $h_1(e) = h_2(e) \oplus hash(\xi_e)$. When inserting an element e, the CF checks both $h_1(e)$ and $h_2(e)$ buckets and stores the fingerprint ξ_e in one of the empty entries. If all entries are occupied, one is randomly chosen and the stored fingerprint is removed and reinserted into its alternate bucket according to Equation (2). This process continues until all elements are successfully inserted into the table or a maximum number of relocations is reached. Although a sequence of relocations may occur, the amortized insertion time is still $O(1)$ [18]. When querying for membership, the CF only needs to probe up to $2b$ entries in two candidate buckets. If a matching fingerprint is found, the query returns TRUE; otherwise, it returns FALSE. The delete operation simply removes the matched fingerprint. Although the CF enables the deletion of elements reasonably, it cannot be expanded or compressed, which may cause low space utilization.

Filters for dynamic data. To address the above limitations, one straightforward way is to periodically reconstruct the filter from the original data stored in slow memory [42, 43]. The Dynamic BF [20] proposes to use a chain of homogeneous BFs, which initially involves one BF and then appends new BFs or merges old ones as needed. However, in the worst case, all BFs need to be checked to answer AMQs, leading to an increase in query costs and errors. Some papers [3, 35] propose using a chain of heterogeneous BFs, where BFs along the chain have larger sizes and tighter maximum error probabilities, which reduces the worst query time to a logarithmic scale. The Dynamic CF [10] extends the chaining architecture to CFs, providing better support for element deletions. The Logarithmic Dynamic CF [45] designs a multi-level tree structure to organize CFs, where the most significant bits of the hash value of the element are used to locate tree nodes, and the remaining bits are used as a fingerprint. This optimizes the space and query costs of Dynamic CF. InfiniFilter [14] is a variant of QF. It expands by doubling in size and transferring bits of the fingerprint to the slot address. By chaining multiple InfiniFilters together, it can adapt to flexible data sizes. A growing-length fingerprint technique is further applied to stabilize error rates. These methods can perform well when the actual data size is close to the estimated size. However, they are not suitable for unbounded streaming data. As the data size grows, their accuracy drops, query cost rises, and storage cost eventually reaches a limit.

Filters for streaming data. Filters for streaming data need to effectively delete stale elements to make room for fresh ones, as there is a dilemma between unlimited data and limited storage. It is different from deletions in dynamic scenarios, where the user initiates the deletion of a specific element. Lots of works have been proposed [8, 13, 15, 16, 19, 26, 34, 38–40, 44]. We detail two representative approaches, *i.e.,* Stable BF [15] and Sliding BF [19], which will be used as our baselines. Stable BF [15] is a variant of BF, consisting of an array of counters. To insert an incoming element, P counters are first randomly selected and decremented by 1 if they are non-zero, and then K independent hash addresses are calculated and the corresponding counters are set to MAX. Querying in Stable BF is similar to that in BF. Intuitively, the probability of an element being removed by iterative random deletions increases as the element becomes older. However, there is a chance that a recent element is deleted in Stable BF, resulting in FNs. Streaming QF [16] adopts a similar random deletion strategy on top of QF. To address the issue of FNs, Decaying BF [38] sets MAX to the sliding window size and decrements all counters by 1 whenever a new element arrives. However, it incurs a large time cost. Sliding BF [19] aims at data stream processing in sliding windows. It maintains an array of buckets divided into k segments, with each bucket containing d fields (bits). To remove outdated information, it designs a scanning operation that goes through the array repeatedly at a constant speed. When the scanning pointer points to a bucket, the value in the

Table 1. List of key notations

Notation	Description
$\mathbb{U}$	The universe with cardinality $\|\mathbb{U}\| = u$
S	A data stream
e, e_i	An element in the stream (at the timestamp i)
W_0, w	The default window size, a user-defined window size
n	The size of the current stream
$S_n(w)$	The sliding window of size w at the current timestamp n
q, ϵ	The queried element, the specified FPR
m	The number of buckets in a CF
b	The number of entries per bucket
α, α^*	The load factor, the optimal load factor
ξ_e	The fingerprint of an element e
f	The fingerprint length in bits
κ	The capacity of a CF in LCF
L	The maximum number of CFs in LCF

field d is deleted, values in other fields are moved backward in sequence, and the field 0 is flushed. Additionally, there are diverse space recycling strategies proposed for streaming filters, such as Double Buffering [8], A2 [44], Forgetful BF [40] and so on. However, the range of information these filters maintain is determined in advance, making them inflexible to various user requirements. For example, their performance would inevitably decay for windows beyond the presupposed range.

Learned filters. In [25], Kraska *et al.* propose the learned Bloom filter (LBF). By regarding the AMQ as a classification problem, LBF combines a pre-trained classifier with a backup BF and eases the space requirement of BF. Some variants, such as sandwiched LBF [32] and adaptive LBF [12], are proposed to optimize FPR further, but they are limited to the static case. Stable LBF [27] is the first work to extend the application scenario of learned filters to streaming data by replacing the backup BF with stable BF [15]. However, stable LBF is not a suitable solution for our problem because: 1) we do not assume the evident existence of classification characteristics in the data, and 2) stable LBF cannot distinguish different window sizes.

Range filters. The range membership query can be considered as a decision version of the range query, which aims to determine whether there is an element in the dataset between values a and b. When $a = b$, it is simplified to the AMQ. Several learned range filters have been proposed [23, 29, 41, 46], but dynamic updates are not considered in these works due to their application scenarios, such as LSM-based key-value stores. Persistent BF [36] and Stair BF [47] are two range filters that can be used to answer AMQs over variable-sized sliding windows. Stair BF [47] divides the stream into periods and constructs homogeneous BFs for each period, with more recent periods having more BFs, like a stair. Each incoming element is inserted into all BFs of the current period. When the memory limit is reached, one BF is deleted from each period. Querying is performed by testing each BF covered by the queried period. Two optimization techniques, vertical sampling and horizontal sharing, are proposed to further improve space utilization and query accuracy. Similar to filters for streaming data, it cannot adapt well to flexible window sizes. When the window size is large, an 'unknown' is returned.

3 PRELIMINARY

In this section, we introduce some basic concepts and formulate the AMQ over variable-sized sliding windows. Frequently used notations are summarized in Table 1.

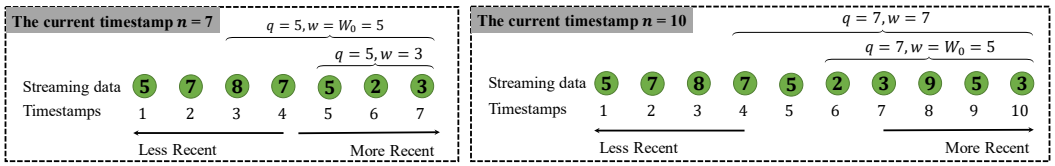


Fig. 1. The AMQ over variable-sized sliding windows

3.1 Concepts

Definition 1 (Data Stream [15]). *Given a universe $\mathbb{U}$ of size u,* i.e., $\mathbb{U} = \{x_1, x_2, \ldots, x_u\}$, *a data stream S is a sequence of elements drawn from $\mathbb{U}$, arranged in chronological order,* i.e., $S = e_1, e_2, \ldots, e_n$, *where n is the size of the current stream and can be infinite. Elements in the universe are different from each other, while the same element may appear more than once in a data stream.*

Definition 2 (Sliding window [19]). *A sliding window of size $w \in \mathbb{Z}^+$ means the most recently seen w elements in a data stream, denoted by $S_n(w) = e_{\max\{1,n-w+1\}}, \ldots, e_n$.*

Definition 3 (Membership query over fixed-size sliding windows [19]). *Given a sliding window size W_0, find out whether a queried element $q \in \mathbb{U}$ appears in $S_n(W_0)$ or not,* i.e., *returns TRUE if $q \in S_n(W_0)$, otherwise FALSE.*

3.2 Problem Statement

This paper generalizes the membership query over fixed-size sliding windows to variable-sized sliding windows, which can be formulated as follows:

Definition 4 (Membership query over variable-sized sliding windows). *For any user-defined window size $w \in \mathbb{Z}^+$ and query $q \in \mathbb{U}$, find out whether $q \in S_n(w)$ or not, denoted as $\textsc{Query}(q, w)$.*

Since we assume that the memory budget M is not large enough to store all elements in the stream, it is difficult to answer the problem precisely. Therefore, our goal is to develop a filter to approximate the answer and minimize the number of errors, including both false positives and false negatives.

Remark 1. *We call a queried element q a false positive (FP) if $q \notin S_n(w)$ but is reported as TRUE. We call a queried element q a false negative (FN) if $q \in S_n(w)$ but is reported as FALSE. We call the probabilities of these errors false positive rate (FPR) and false negative rate (FNR), respectively.*

Example 1. *Consider a stream $S = 5, 7, 8, 7, 5, 2, 3, 9, 5, 3, \ldots$ drawn from a universe $\mathbb{U} = \{1, 2, \ldots, 9\}$, where each data element is represented by a value for simplicity, as shown in Figure 1. Under the conventional sliding window setting, the window size W_0 is fixed before the filter is built, say $W_0 = 5$. As time goes by, the window slides and always keeps the most recent 5 elements. Users can ask whether q is in $\{8, 7, 5, 2, 3\}$ when the current timestamp n is 7, and whether q is in $\{2, 3, 9, 5, 3\}$ when n is 10, where $q \in \mathbb{U}$ is any queried element. Under the variable-sized sliding window setting, the window size w is ad hoc, which is specified by users during the query. For example, users can also ask for the presence of q among the most recently observed 3 or 7 elements by specifying $w = 3$ or 7, respectively.*

4 THE BASIC LCF (LCF_B)

In this section, we begin with a basic LCF (denoted by LCF_B). To improve space efficiency, we develop a compact LCF (denoted by LCF_C) by incorporating two space optimization strategies, to be introduced in Section 5. Through theoretical analysis and parameter optimizations, we further

propose an optimized LCF (denoted by LCF_O) in Section 6, which improves query accuracy and achieves a better accuracy/storage trade-off.

4.1 Intuition

To provide specific judgments of the presence of queried elements regarding different window sizes, the data stream can be managed in consecutive non-overlapping blocks. Only the blocks covered by the sliding window will be checked during the query stage. In this work, we use the CF to manage the elements in each block due to the consideration of subsequent space optimization in LCF_C. Apparently, traditional CFs alone cannot handle the case with an arbitrary window size w since the CFs have finite space limits but w can be very large. To address this issue, we draw inspiration from the learned index and introduce a machine learning model, *also known as* the oracle $\mathcal{M}$, to assist traditional CFs. The oracle $\mathcal{M}$ can learn the frequency characteristics of the data in the stream, allowing it to estimate the time interval between two consecutive occurrences of a specific item. For query windows larger than the range taken care of by the CFs, we further consult $\mathcal{M}$ to assist the decision-making process. The non-member decision is made if and only if CFs and $\mathcal{M}$ share the same FALSE decision. Otherwise, the member decision will be returned.

4.2 Data structure

LCF_B consists of two parts: a list of CFs and a learned oracle.

4.2.1 The CF chaining. The first component of LCF_B is a list of homogeneous CFs, which are constructed and deconstructed dynamically. Initially, there is only one CF, and then additional filters are appended to accommodate data growth through Insert operations, to be introduced in Section 4.3.1. Let L represent the maximum number of CFs that can be stored with a specific memory allocation. To make room for incoming elements, LCF_B iteratively deletes old CFs, ensuring the space usage never exceeds the L limit.

4.2.2 The learned oracle. The other component of LCF_B is an oracle $\mathcal{M}$, which aims to learn the distribution of gaps. A gap means the number of timestamps between an element and its last duplicate. For example, the element $e_{10} = 3$ and 3 was last seen at timestamp 7, *i.e.,* $e_7 = e_{10} = 3$, then the gap of element 3 is $10 - 7 = 3$. The frequency of an element in the data stream is inversely related to the gap between its occurrences. Elements with higher frequency have smaller gaps. Thus, we equivalently estimate the gap distribution by estimating the frequency distribution of elements, which is a fundamental task in data science. Extensive frequency estimation algorithms have been proposed, including both learning-based algorithms [4, 21, 33] and their traditional counterparts [9, 11, 17]. In this work, we choose a learning-based algorithm [21] as the oracle due to its ability to achieve better performance by leveraging useful patterns in the input data. The basic idea is to use a subset S' of the data stream S to learn the potential properties that bestow frequency to certain elements, thus enabling the oracle to estimate the frequency of elements, even if they did not appear in the training set S'. For example, in the text data, the oracle would learn that shorter words tend to be more frequent. Let v_x and $\tilde{v}_x$ represent the frequency (the number of times x occurs in S') and the estimated frequency, respectively. This leads to the expected estimation error as follows:

$$\mathcal{L} = 1/N \sum_{x \in S'} |\tilde{v}_x - v_x| \cdot v_x, \tag{3}$$

where N is the sum of all frequencies. We train the neural network to predict the frequency and minimize the squared loss of the prediction. The model is trained before the Insert (Update) operation and used as the oracle in the Query and Update operations as stated below.

Algorithm 1: INSERT

Input: e: a newly arrived element;
Output: The LCF_B $\mathcal{F}$;

```
1 cnt ← # of elements arrived since the creation of the F[1];
2 if cnt < κ and F[1] does not have ξ_e then
3     insert e into F[1];
4 if insertion failure or cnt = κ then
5     Append a new CF to F;
6     insert e into the new CF ;
7     cnt ← 0;
8 cnt ← cnt + 1;
9 return F
```

Algorithm 2: QUERY

Input: q: a queried element; w: the window size;
Output: TRUE or FALSE

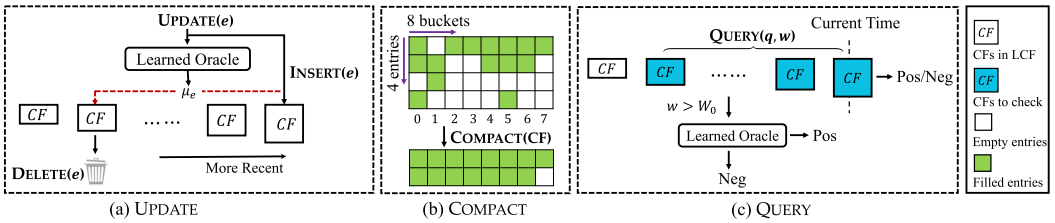
```
1 s ← # of CFs covered by the sliding window;
2 for i = 1 to s do
3     if F[i] has ξ_q then
4         return TRUE ;
5 if w > W_0 then
6     μ_q ← M(q);
7     if W_0 < μ_q ≤ w then
8         return TRUE;
9 return FALSE;
```

4.3 Operations

4.3.1 INSERT. Algorithm 1 presents the INSERT operation of LCF_B in detail. To simplify expression, we always refer to the newest CF as $\mathcal{F}[1]$, and the larger numbered CF contains older elements. For each incoming element e, we first check whether the $\mathcal{F}[1]$ is full, *i.e.,* whether the number of elements that arrived since this CF was created is no less than a threshold κ (Line 2). If not, *i.e.,* $cnt < \kappa$, a standard CF query of e, as described in Section 2, is performed in $\mathcal{F}[1]$ (Line 2) and if the $\mathcal{F}[1]$ does not have ξ_e, the algorithm performs a standard CF insert operation in the $\mathcal{F}[1]$ (Line 3). If the insertion succeeds, we increment cnt by one (Line 8); otherwise, a new CF is created in which e is inserted and cnt is set to 0 (Line 5 - 7). Such initialization also needs to be done when the CF to be inserted is already full, *i.e.,* $cnt = \kappa$ (Line 4). For the case where the $\mathcal{F}[1]$ is not full and already contains ξ_e, the situation is simple. We directly increment cnt by 1 since we only store one copy of each fingerprint (Line 8).

4.3.2 QUERY. Algorithm 2 and Figure 2(c) outline the QUERY operation of LCF_B. Let l be the number of CFs in the $\mathcal{F}$ and s be the number of CFs covered by the sliding window. We have:

$$s = \begin{cases} \min\{l, \lceil \frac{w}{\kappa} \rceil\}, & cnt = \kappa, \\ \min\{l, \lceil \frac{w-cnt}{\kappa} \rceil + 1\}, & 1 \le cnt < \kappa. \end{cases} \tag{4}$$



(a) UPDATE (b) COMPACT (c) QUERY

Fig. 2. An overview of the compact LCF

LCF_B looks through all s CFs and performs a standard CF query in each CF. Once a matched fingerprint is found, the algorithm terminates immediately and returns TRUE (Line 2 - 4). Otherwise, if the window size $w \leq W_0$, FALSE is returned directly. However, if $w > W_0$ (Line 5), FNs may occur due to the necessary deletion of old CFs to bound the storage cost within a given amount. As w increases, the number of FNs increases. To control errors of large window sizes, we resort to the model for a more reliable decision. Initially, we compute the gap of q, denoted as μ_q, by invoking the oracle (Line 6). If $\mu_q \leq W_0$, it implies that the model recognizes $q \in S_n(W_0)$, which is inconsistent with the result provided by the CFs. In this case, we prioritize the decision of the CFs because they can guarantee no FN error, *i.e.,* FALSE is returned (Line 9). If $\mu_q > w$, the model perceives q as infrequent and infers that $q \notin S_n(w)$. As both the CFs and model make a non-member decision, FALSE is returned (Line 9). Otherwise, *i.e.,* $W_0 < \mu_q \leq w$ (Line 7), the model thinks $q \in S_n(w)$. Although the CFs indicate that $q \notin S_n(W_0)$, they lack any information in $S_n(w) \backslash S_n(W_0)$, which is part of the query window. Therefore, we challenge the negative output from the CFs and return TRUE (Line 8).

5 THE COMPACT LCF (LCF_C)

LCF_B can answer AMQs over variable-sized sliding windows but it is not space-efficient. Together with the space overhead of the model, the space cost of LCF_B is greater than other filters for streaming data. Therefore, in this section, we further propose two space optimization strategies, by which we only maintain what has to be preserved and remove what can be forgotten.

5.1 Intuition

OBSERVATION 1. *The answer to a membership query of $q \in \mathbb{U}$ over sliding windows of any size is determined by q's latest arrival time. If the latest arrival time of q is larger than $n - w$, then q is a member; otherwise it is a non-member.*

EXAMPLE 2. *Reconsider the data stream in Example 1. For the membership QUERY$(5, 3)$, where $q = 5$ and the window size $w = 3$, issued at $n = 7$, we know the answer should be TRUE since $n - w = 7 - 3 = 4$ is smaller than its latest arrival time* (i.e.,5). *Similarly, as time progresses to $n = 10$, the result of QUERY$(7, 5) =$ FALSE since $n - w = 5 > 4$.*

Based on the above, we can conclude that only the most recent occurrence of an item determines the results of AMQs, whereas historical records do not contribute to the precision. Therefore, it would be beneficial to lower the space consumption of LCF_B by eliminating redundant historical records from the filter, especially when the input data conforms to specific distributions. Instead of merely discarding elements that fall outside the designated window, LCF_C is expected to remove the redundant data selectively from the filter without compromising precision.

One straightforward deletion strategy is traversing all CFs and performing CF delete operations if duplicate fingerprints are found. However, it will incur a large time cost. Inspired by the learned B$^+$-tree [25] where positioning can be processed by a function invocation in $O(1)$ time, we consider

Algorithm 3: UPDATE

Input: e: a newly arrived element;
Output: The LCF_C $\mathcal{F}$;
/* Replace lines 2-3 in Algorithm 1 with the following, all else being identical */
1 **if** $cnt < \kappa$ *and* $\mathcal{F}[1]$ *does not have* ξ_e **then**
2 **for** $i = t_1(e)$ *to* $t_2(e)$ **do**
3 delete ξ_e from the i-th CF in $\mathcal{F}$;
4 insert e into $\mathcal{F}[1]$;

an UPDATE operation for LCF_C. Different from LCF_B that only inserts newly arrived elements in the new CF, LCF_C accompanies each insertion of e in a new CF with a deletion of e in a stale CF powered by the learned oracle. Thus, the CF that most likely contains the old version of an element can be located efficiently. Figure 2(a) illustrates the idea of UPDATE. As elements in old CFs are continuously deleted during the UPDATES, the empty entries in old CFs increase, so it is necessary to develop a space reclaimer to release vacant space. Therefore, we develop a COMPACT operation to rearrange the fingerprints stored in sparse CFs and free up the space, as shown in Figure 2(b). The QUERY operation in LCF_C is the same as that in LCF_B, as discussed in Algorithm 2 and illustrated in Figure 2(c). The deletions during the UPDATES will not incur FNs, as proved in Theorem 5, Section 6. Therefore, by setting the parameters appropriately, LCF_C is a promising way to yield a new storage/accuracy balance point, to be described in Section 6. The data structure of LCF_C is similar to that of LCF_B, *i.e.,* a list of compressible CFs and a learned oracle.

5.2 Update

We simplify the presentation by omitting identical components in the UPDATE operation of LCF_C and the INSERT operation of LCF_B, as shown in Algorithm 3. For each incoming element e, we first check whether $\mathcal{F}[1]$ is full or whether it already contains ξ_e (Line 1). If not, the algorithm invokes the learned oracle $\mathcal{M}$, to predict the gap μ_e of e. According to the estimated gap μ_e, the search range of e's last duplicate is whittled down from the $t_1(e)$-th CF to the $t_2(e)$-th CF as follows:

$$t_1(e) = \left\lceil \frac{\mu_e - cnt}{\kappa} \right\rceil + \gamma_1, \tag{5}$$

$$t_2(e) = \left\lceil \frac{\mu_e - cnt}{\kappa} \right\rceil - \gamma_2, \tag{6}$$

where $\gamma_1, \gamma_2 \in \mathbb{N}$ refer to the number of additional CFs checked. The intuition behind this is from the learned B$^+$-tree that corrects the model error by searching a range of $[pos - min_err, pos + max_err]$, where pos is the predicted position, min_err and max_err are the worst under- and over-prediction of a position. The values of γ_1 and γ_2 affect the update efficiency and space consumption. Intuitively, the larger γ_1 and γ_2, the larger range we search during the UPDATE (that affects the efficiency) and the more likely to find and remove redundancy (that affects the space consumption). We delete ξ_e in these CFs (if exist) by performing standard CF delete operations (Line 2 - 3). After erasing old information, the algorithm updates $\mathcal{F}$ by performing a standard CF insertion in the $\mathcal{F}[1]$ (Line 4).

5.3 Compact

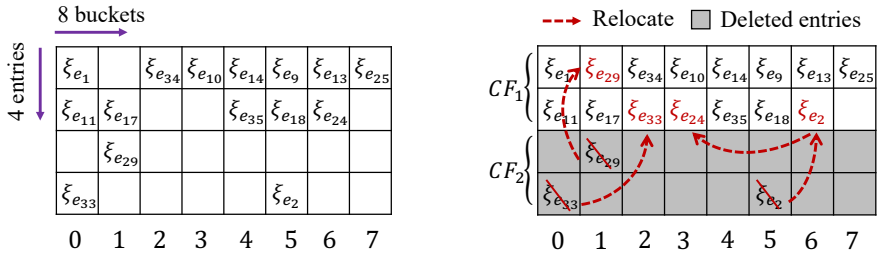
The load factor α is an important metric of space efficiency for CFs, which is defined as the proportion of filled entries in a CF, *i.e.,* $\alpha = \frac{\#\text{ of fingerprints}}{\#\text{ of entries}}$, and $\alpha^* \approx 95\%$ is the empirical optimal

Algorithm 4: COMPACT

Input: CF: a cuckoo filter in $\mathcal{F}$; b': a compaction parameter;
Output: A compacted CF or False;

```
1 CF1 ← a CF with each bucket containing the first b' entries;
2 CF2 ← a CF with each bucket containing the rest entries;
3 for each non-empty bucket in CF2 do
4     insert ξe's into CF1;
5     if CF1 is full and CF2 is non-empty then
6         return FALSE ;
7 return CF1
```

value in the default setting [18]. We design a COMPACT operation for LCF_C to keep a high α, as presented in Algorithm 4. For clarity, we call the cuckoo hash table with m buckets, each containing the first b' entries CF_1 and the remaining $b - b'$ entries CF_2 (Line 1 - 2). As empty entries resulting from deletions during the UPDATE process are dispersed throughout the cuckoo filter, we reorganize the fingerprints to achieve optimal compression. Specifically, we relocate fingerprints in CF_2 to the corresponding locations in CF_1 by performing standard CF insert operations (Line 3 - 4). If the insertion fails before emptying CF_2, the COMPACT fails (Line 5 - 6). Otherwise, we release the last $m \cdot (b - b')$ empty entries and return CF_1 as the compacted CF (Line 7). The load factor of the compacted CF climbs back close to α^* since $\alpha' \cdot mb' = \alpha \cdot mb$, *i.e.*, $\alpha' = \frac{\alpha b}{b'} = \frac{\alpha b}{\lceil b \cdot \frac{\alpha}{\alpha^*} \rceil} \approx \frac{\alpha b}{b \cdot \frac{\alpha}{\alpha^*}} = \alpha^*$. Here we use α and α' to represent the load factors before and after the compact to simplify expressions. Obviously, if all entries in an old CF are emptied, we can directly discard it to release space.



(a) A CF before COMPACT (b) A CF after COMPACT

Fig. 3. An example of the COMPACT operation

EXAMPLE 3. *Figure 3 gives an example of the* COMPACT *operation where* $m = 8, b = 4, b' = 2$. *Suppose that the CF contains* 15 *fingerprints at present, as shown in Figure 3(a). The load factor* $\alpha = 15/32 \approx 0.469 < b'/b \cdot \alpha^* = 1/2 \cdot 0.95 = 0.475$, *thus we conduct* COMPACT. *There are* 3 *fingerprints in* CF_2, i.e., $\xi_{e_{33}}$, $\xi_{e_{29}}$ *and* ξ_{e_2}. *For* $\xi_{e_{33}}$, *suppose that* $h_1(e_{33}) \oplus hash(\xi_{e_{33}}) = 2$. *Since the counterpart bucket in* CF_1 (i.e., *bucket* 0) *is fully occupied yet the alternative bucket* 2 *has an empty entry, we insert* $\xi_{e_{33}}$ *to the bucket* 2. *For* $\xi_{e_{29}}$, *it is directly relocated to the empty entry in its counterpart bucket in* CF_1 (i.e., *bucket* 1). *For* ξ_{e_2}, *suppose that* $h_1(e_2) \oplus hash(\xi_{e_2}) = 6$. *Since both of its candidate buckets are full,* $\xi_{e_{24}}$ *is randomly chosen and kicked out to its alternate bucket,* e.g., *bucket* 3, *and* ξ_{e_2} *is stored in the bucket* 6. *No insert failure occurs, so we free up the space occupied by* CF_2, *for a total of* $m \cdot (b - b') = 16$ *empty entries. After the* COMPACT *operation, the space utilization climbs back up to* $15/16 \approx 94\%$.

6 THE OPTIMIZED LCF (LCF_O)

LCF_C is a promising solution to handle flexible query windows at a smaller space cost, but how to configure it to meet the user's need for accuracy is still challenging. In this section, we analyze its expected FPR, FNR, and complexities, regarding which we optimize parameters to achieve satisfactory performance.

In the analysis presented hereafter, we adopt the following 3 assumptions: 1) A new CF is created due to $cnt = \kappa$. It is easy to satisfy by properly dimensioning the CF [18]. 2) Each element in the data stream is drawn independently and W_0, w are multiples of κ to simplify the expression. 3) We simply consider a uniform query distribution since we do not assume a known realistic query workload. For other distributions, the conclusions also apply with only slight differences in the formulas of the expected FPR/FNR.

We begin by introducing a lemma that is used to simplify the subsequent proofs. Simply speaking, it provides an approximation to the occurrence probability of the fingerprint in data streams from the perspective of expectation.

Lemma 1. *Let an element $x_i \in \mathbb{U}$ be drawn with probability p_i satisfying $\sum_{i=1}^{u} p_i = 1$, then the expected probability of a fingerprint appearing in data streams is $\frac{1}{2^f}$.*

Proof. Let the probability of a fingerprint ξ appearing in data streams be $\theta(\xi)$. Suppose that the fingerprint of $x_{i_1}, x_{i_2}, \ldots, x_{i_g}$ is ξ, then we have $\theta(\xi) = p_{i_1} + \cdots + p_{i_g}$ with a probability of $(\frac{1}{2^f})^g(1-\frac{1}{2^f})^{u-g}$. Therefore, the expectation of $\theta(\xi)$ is as follows:

$$\begin{aligned}\mathbb{E}[\theta(\xi)] &= \sum_{g=1}^{u}\left(\frac{1}{2^f}\right)^g\left(1-\frac{1}{2^f}\right)^{u-g}\cdot\sum_{(i_1,\ldots,i_g)\in D_g} p_{i_1}+\cdots+p_{i_g} \\ &= \sum_{g=1}^{u}\binom{u-1}{g-1}\left(\frac{1}{2^f}\right)^g\left(1-\frac{1}{2^f}\right)^{u-g} = \frac{1}{2^f}.\end{aligned} \tag{7}$$

where D_g is a set of g-element subsets from $\mathbb{U}$. □

6.1 False Positive Rate

6.1.1 $w \le W_0$.

Theorem 1. *The probability that a FP occurs when conducting a query(q, w) using LCF_C is*

$$\Pr[FP_{q,w}] \le 1-(1-p_i)^{\kappa}\left(1-\frac{1}{2^f}\right)^{2\bar{b}\left(\frac{w}{\kappa}+1\right)}, \tag{8}$$

where p_i is the probability of $q(=x_i)$ drawn, $\bar{b}$ is the average b of CFs covered by the sliding window.

Proof. We decompose a sliding window into 3 parts $w = cnt + (s-2)\cdot\kappa + w'$, where $cnt \in [1, \kappa)$ is the number of elements in the $\mathcal{F}[1]$, s is the number of CFs that is covered by the sliding window as Equation (4), and $w' \in [1, \kappa)$ is the number of elements in the $\mathcal{F}[s]$. For any queried element $q \notin S_n(w)$, Query(q, w) returns either TRUE or FALSE. If a TRUE answer is returned, then q is a FP, otherwise, q is a true negative (TN). Therefore, we have $\Pr[FP_{q,w}] = 1 - \Pr[TN_{q,w}]$. In what follows, we derive $\Pr[TN_{q,w}]$ instead.

$$\begin{aligned}\Pr[TN_{q,w}] &= \Pr[\text{Query}(q,w) = \text{FALSE} | q \notin S_n(w)] \\ &= \frac{(1-p_i)^{(s-1)\kappa+cnt}\cdot\prod_{i=1}^{s}(1-\theta(\xi_q))^{2b_i\alpha_i}}{(1-p_i)^w} \\ &\ge \frac{(1-p_i)^{(s-1)\kappa+cnt}\cdot\prod_{i=1}^{s}(1-\theta(\xi_q))^{2b_i}}{(1-p_i)^w},\end{aligned} \tag{9}$$

where b_i and α_i are the number of entries each bucket contains and the load factor of $\mathcal{F}[i]$, respectively. According to Lemma 1, we substitute $\theta(\xi_q)$ with $\frac{1}{2^f}$, and thus

$$\begin{aligned}\Pr[TN_{q,w}] &\geq \frac{(1-p_i)^{(s-1)\kappa+cnt}\cdot\prod_{i=1}^{s}\left(1-\frac{1}{2^f}\right)^{2b_i}}{(1-p_i)^{w}}\\ &\geq (1-p_i)^{\kappa-w'}\cdot\left(1-\frac{1}{2^f}\right)^{2\bar{b}s}\\ &\geq (1-p_i)^{\kappa}\cdot\left(1-\frac{1}{2^f}\right)^{2\bar{b}\left(\frac{w}{\kappa}+1\right)}.\end{aligned} \tag{10}$$

Therefore, $\Pr[FP_{q,w}] = 1-\Pr[TN_{q,w}] \leq 1-(1-p_i)^{\kappa}(1-\frac{1}{2^f})^{2\bar{b}(\frac{w}{\kappa}+1)}$. □

Corollary 1. *The upper bound of the probability that a FP occurs over a w-size sliding window monotonically increases as w increases.*

Theorem 2. *The expected FPR over a w-width sliding window is*

$$\mathbb{E}[FPR_w] \leq 1-\frac{1}{u}\sum_{x_i\in\mathbb{U}}(1-p_i)^{\kappa}(1-\frac{1}{2^f})^{2\bar{b}(\frac{w}{\kappa}+1)}. \tag{11}$$

Proof. This is obtained by applying the formula of expectation $\mathbb{E}[FPR_w] = \frac{1}{u}\sum_{x_i\in\mathbb{U}}\Pr[FP_{x_i,w}]$ and the result in Theorem 1. □

Corollary 2. *Since the function $\Pr[FP_{x_i}, w]$ is continuous, there must be an average $\hat{p}\in(0,1)$ such that*

$$\mathbb{E}[FPR_w] \leq 1-(1-\hat{p})^{\kappa}\left(1-\frac{1}{2^f}\right)^{2\bar{b}\left(\frac{w}{\kappa}+1\right)}. \tag{12}$$

In what follows, we use this formula to simplify the expression.

6.1.2 $w > W_0$.

As per Algorithm 2, both the CFs and model may lead to FPs when $w > W_0$. TRUE is returned if a matched fingerprint is found in a CF covered by the sliding window or none of the CFs have a matched fingerprint but $W_0 < u_q \leq w$. Thus, the total probability of a FP can be estimated as the sum of two parts as follows:

Theorem 3. *The probability that a FP occurs when conducting a QUERY(q,w) using LCF_C is*

$$\Pr[FP_{q,w}] \leq \Pr_{\mathcal{M}}[FP] + \left(1-\Pr_{\mathcal{M}}[FP]\right)\cdot\left(1-(1-p_i)^{\kappa}\left(1-\frac{1}{2^f}\right)^{2\bar{b}L}\right), \tag{13}$$

where $L = \frac{W_0}{\kappa}+1$ and $\Pr_{\mathcal{M}}[FP] = \Pr[W_0 < \mu_q \leq w | q \notin S_n(w)]$.

Proof. We have $\Pr[FP_{q,w}] = \Pr_{\text{CFs}}[FP] + \Pr_{\text{CFs}}[TN]\cdot\Pr_{\mathcal{M}}[FP] = \Pr_{\mathcal{M}}[FP] + (1-\Pr_{\mathcal{M}}[FP])\cdot\Pr_{\text{CFs}}[FP]$, where $\Pr_{\text{CFs}}[FP]$ and $\Pr_{\mathcal{M}}[FP]$ are the probabilities that a FP is caused by the CFs and the model, respectively. Based on Corollary 1, $\Pr_{\text{CFs}}[FP] \leq (1-(1-p_i)^{\kappa}(1-\frac{1}{2^f})^{2\bar{b}L}$ given that we maintain at most $L = \frac{W_0}{\kappa}+1$ CFs. In sum, we obtain the upper bound on the probability of FP as Equation (13). □

Theorem 4. *The expected FPR over a w-width sliding window is*

$$\mathbb{E}[FPR_w] \leq F_p + (1-F_p)\cdot\left(1-(1-\hat{p})^{\kappa}\left(1-\frac{1}{2^f}\right)^{2\bar{b}L}\right), \tag{14}$$

where $F_p = \frac{1}{u}\sum_{x_i \in \mathbb{U}} \Pr_{\mathcal{M}}[FP]$ *is the expected FPR of model.*

6.2 False Negative Rate

6.2.1 $w \le W_0$.

Theorem 5. *There is no false negative error.*

Proof. According to the dynamic allocation strategy, Update and Compact operations, LCF_C represents at least the recent W_0 elements and at most the recent $W_0 + \kappa$ elements, depending on current timestamps, which is a super set of the W_0-size sliding window. For any element $x \in S_n(w)$, if it is not deleted during the Update, it obviously cannot be a FN. If x is deleted because it appears again, x will also be inserted into the newest CF at the time according to Algorithm 3. Due to the existence of a newer copy of x in the CF within the sliding window, it cannot produce a FN. If x is wrongly deleted when Updating a different element y sharing the same buckets and fingerprint with x, x still cannot be a FN. This is because y is inserted in a newer CF to serve the same role as x. For example, x is removed from the CF_x and y is inserted into the newest CF_y at the time when Updating y. During the query phase, Algorithm 2 performs standard CF queries along the CF chain from newest to oldest. When it encounters CF_y, TRUE is returned because a matched fingerprint $\xi_y (= \xi_x)$ is found. □

6.2.2 $w > W_0$.

When $w > W_0$, FN errors may arise due to the inevitable deletion of stale CFs and inaccuracy in the oracle. According to Algorithm 2, FALSE is returned if both the CFs and model make a non-member decision. Therefore, the total probability of a FN can be estimated as the product of two parts as follows:

Theorem 6. *The probability that a FN occurs when conducting a* query(q, w) *using LCF_C is*

$$\Pr[FN_{q,w}] \approx \frac{(1-p_i)^{W_0} - (1-p_i)^{w}}{1-(1-p_i)^{w}} \cdot \left(1 - \frac{1}{2^f}\right)^{2\bar{b}L} \cdot Pr_{\mathcal{M}}[FN], \tag{15}$$

where $\Pr_{\mathcal{M}}[FN] = \Pr[w < \mu_q | q \in S_n(w) \backslash S_n(w')]$.

Proof. Let $w' = W_0 + cnt$ denote the number of elements maintained in CFs. If $q \in S_n(w')$, there cannot be any FN error caused by deletions during the Update operations according to Theorem 5. Thus, we consider the case $q \in S_n(w) \backslash S_n(w')$ in what follows:

$$\begin{aligned} \Pr[FN_{q,w}] &= \Pr[\text{query}(q,w) = \text{FALSE} | q \in S_n(w)] \\ &= \frac{(1-p_i)^{w'}(1-(1-p_i)^{w-w'}) \cdot \prod_{i=1}^{L}\left(1-\frac{1}{2^f}\right)^{2b_i\alpha_i} \cdot \Pr_{\mathcal{M}}[FN]}{1-(1-p_i)^{w}} \\ &\approx \frac{(1-p_i)^{W_0} - (1-p_i)^{w}}{1-(1-p_i)^{w}} \cdot \left(1-\frac{1}{2^f}\right)^{2\bar{b}L} \cdot \Pr_{\mathcal{M}}[FN]. \end{aligned} \tag{16}$$

This completes the proof. □

Theorem 7. *The expected FNR over a w-width sliding window is*

$$\mathbb{E}[FNR_w] \approx \frac{(1-\hat{p})^{W_0} - (1-\hat{p})^{w}}{1-(1-\hat{p})^{w}} \cdot \left(1 - \frac{1}{2^f}\right)^{2\bar{b}L} \cdot F_n, \tag{17}$$

where $F_n = \frac{1}{u}\sum_{x_i \in \mathbb{U}} \Pr_{\mathcal{M}}[FN]$ *is the expected FNR of the model and* $0 < \hat{p} < 1$ *exists due to the continuity of function.*

6.3 Parameter Settings

The parameters include the length of a data stream to sketch W_0, the capacity of a CF κ, the number of CFs L, and the specification for each CF, *i.e.,* b, m and f. The users are enabled to provide their desired query q, window size w, and the acceptable FPR ϵ. Then, we need to choose a combination of these parameters for LCF_O that minimizes the expected FPR and FNR under the conditions that when $w \leq W_0$, there is no FN and the FPR is smaller than ϵ given limited space M.

6.3.1 *Setting of κ.*

Theorem 8. *Given a sliding window size W_0, the upper bound of the expected FPR of CFs is minimized when $\kappa = \sqrt{\frac{2\bar{b}W_0}{2^f \hat{p}}}$.*

Proof. From the results in Section 6.1, we have

$$\begin{aligned}
\mathbb{E}_{\text{CFs}}[FPR] &\leq 1 - (1-\hat{p})^{\kappa} \left(1 - \frac{1}{2^f}\right)^{2\bar{b}L} \\
&= 1 - e^{-\left[\kappa \ln (1-\hat{p})^{-1} + \frac{2\bar{b}W_0}{\kappa} \ln \left(1-\frac{1}{2^f}\right)^{-1}\right] - 2\bar{b} \ln \left(1-\frac{1}{2^f}\right)^{-1}} \\
&= 1 - e^{-2\sqrt{2\bar{b}W_0 \ln (1-\hat{p})^{-1} \ln \left(1-\frac{1}{2^f}\right)^{-1}} - 2\bar{b} \ln \left(1-\frac{1}{2^f}\right)^{-1}} \\
&\approx 1 - e^{-2\sqrt{\frac{2\bar{b}W_0\hat{p}}{2^f}} - \frac{2\bar{b}}{2^f}}.
\end{aligned} \tag{18}$$

where the second equality holds i.f.f $\kappa = \sqrt{\frac{2\bar{b}W_0}{2^f \hat{p}}}$ according to the inequality of the arithmetic and geometric means. □

Remark 2. *Since it may be hard to estimate $\hat{p}$, an alternative is to use $\frac{1}{u}$ to replace $\hat{p}$,* i.e., $\kappa = \sqrt{\frac{2\bar{b}W_0 u}{2^f}}$. *According to Jensen's inequality,* $1 - e^{-2\sqrt{\frac{2\bar{b}W_0\hat{p}}{2^f}} - \frac{2\bar{b}}{2^f}} \leq 1 - e^{-2\sqrt{\frac{2\bar{b}W_0}{2^f u}} - \frac{2\bar{b}}{2^f}}$.

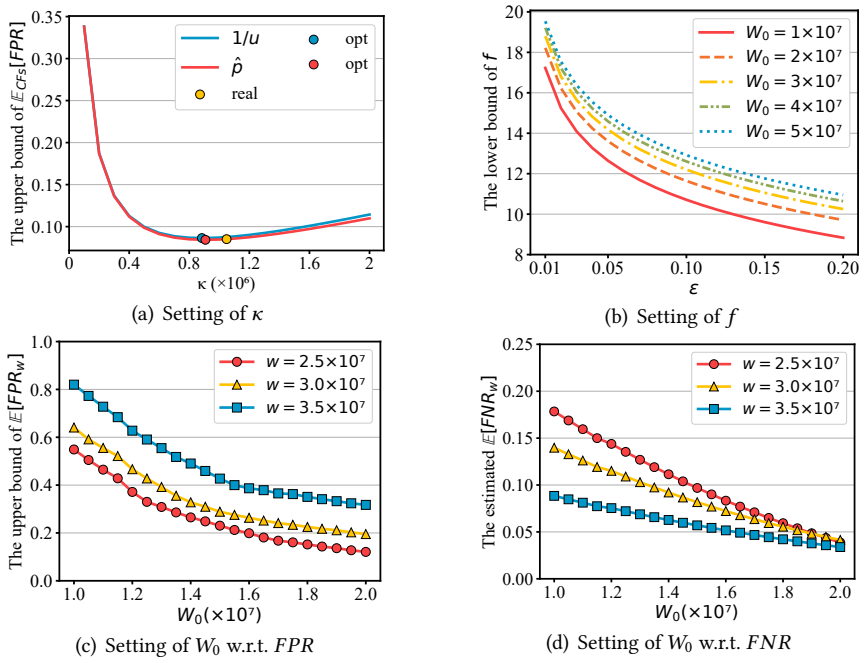
Figure 4(a) gives an example of the upper bound of $\mathbb{E}_{\text{CFs}}[FPR]$ for various κ, where elements in the data stream are chosen independently from a universe with a Zipf distribution [37], $u = 20,000,000$, $\hat{p} = 4.74469 \times 10^{-8}$, $f = 12$, $\bar{b} = 4$ and $W_0 = 20,000,000$ (See Section 7.1 for more details). From Theorem 8, the optimal $\kappa = 907352$, in which case the upper bound is minimized to 0.084, as highlighted by the red circle. By replacing $\hat{p}$ with $\frac{1}{u}$, we get a similar curve, which is consistent with the conclusion in Remark 2. Thus, if W_0, f, b are solved, κ can be solved, and thus $L = W_0/\kappa + 1$ is solved.

6.3.2 *Setting of b, m.*

Following [18], we choose $b = 4$ by default. Since the same element may appear more than once in a data stream and we only store one copy of each element, m should be properly set to achieve a load of 95% after seeing κ elements. The setting of m can be modeled as a variant of the *Coupon Collector's Problem.* Let X_i be the additional number of elements that we need to wait to pass from $i-1$ to i filled entries in a CF. Then, we have

$$\mathbb{E}[\kappa] = \mathbb{E}[\sum_{i=1}^{95\%bm} X_i] = \sum_{i=1}^{3.8m} \frac{m2^{f-1}}{m2^{f-1} - i + 1} \approx m2^{f-1} \ln \frac{2^{f-1}}{2^{f-1} - 3.8} \tag{19}$$

where we assume $p_i = \frac{1}{u}$ for simplicity. It is easy to prove the function $F(f) = 2^{f-1} \ln \frac{2^{f-1}}{2^{f-1}-3.8}$ is monotonically decreasing when $6 \leq f \leq 20$ and $3.8 < F(f) < 4.05$, so we approximate $F(f)$ by 4, and hence m can be set to κ/b. In practice, the distributions may be skewed, resulting in the ideal m



(a) Setting of κ (b) Setting of f

(c) Setting of W_0 w.r.t. *FPR* (d) Setting of W_0 w.r.t. *FNR*

Fig. 4. Theory Verification

deviating from the above estimation. For example, to be introduced in Section 7.3.3, the empirical load factors are 92% and 65% on Gauss and Zipf datasets after inserting κ elements, respectively. That is, the ideal m should have been set to $\kappa/b \cdot \frac{92\%}{95\%}$ and $\kappa/b \cdot \frac{65\%}{95\%}$, respectively. However, we argue that the guidance of m under the assumption of $p_i = 1/u$ is still meaningful. On the one hand, we cannot directly let m be its theoretical optimum. Instead, we need to round m to its closest power of 2, requested by the CF, and then derive κ. The yellow circle in Figure 4(a) gives an example of the actual setting of κ. On the other hand, the over-allocated space can be quickly reclaimed by the Compact operations.

6.3.3 Setting of f. According to Equation (18), increasing the value of f will greatly reduce the FPR, while the growth of W_0 leads to an increase of the FPR. Given W_0 and ϵ, indicating the acceptable FPR within W_0-width sliding window, the FPR constraint can be satisfied by properly choosing f. Specifically, to ensure $1 - exp(-2\sqrt{2\bar{b}W_0\hat{p}/2^f} - 2\bar{b}/2^f) \leq \epsilon$, the minimal f required is:

$$f \geq \left\lceil \log_2 \frac{2\bar{b}}{\epsilon^2} \left(2\hat{p}W_0 + \epsilon + 2\sqrt{W_0\hat{p}(W_0\hat{p} + \epsilon)} \right) \right\rceil. \tag{20}$$

Figure 4(b) visualizes the above equation for various W_0 and ϵ.

6.3.4 Setting of W_0. The value of W_0 depends on the specific application, with constraints of accuracy, response time, and memory budget. For instance, in the recommendation system, W_0 can be several weeks, while in Internet traffic, W_0 might be several minutes. Within the specified ranges, larger values of W_0 can lead to lower error rates. Intuitively, the more information the filter sketches, the more accuracy it can achieve. From Equation (14) and $\Pr_{\mathcal{M}}[FP]$, we can derive that the larger W_0, the smaller $\Pr_{\mathcal{M}}[FP]$, and hence the smaller the expected FPR. We observe the same monotonic decreasing tendency in the expected FNR with increasing W_0. Figure 4(c) and Figure 4(d) confirm our observation, where we simulate the error rates of the model for various W_0 values

under different window sizes $w > W_0$ on the Gauss dataset, to be introduced in Section 7.1, and calculate the corresponding FPR/FNR of the CFs with κ, L, b, m and f set to their optimal values upon the above discussion. However, a large W_0 incurs large space costs. Specifically, the space cost of CFs in LCF_O is $f \cdot$ average # of entries $\cdot$ # of CFs $\leq f \cdot bm \cdot L \approx f\kappa L \approx fW_0$. Given limited space M, we need to ensure that $fW_0 \leq M$. Considering that modern main memory is generally abundant compared to the space consumption of LCF_O (several MB), memory budget is not a big concern. Meanwhile, a large W_0 incurs large time costs, as a longer fingerprint is required to keep the error rate lower than a user-specific ϵ based on Equation (20). If higher time costs can be tolerated or relieved by hardware acceleration, larger values of f can be applied to pursue higher accuracy over larger default query windows.

6.3.5 Summary. The formulas derived above allow for sizing CFs to meet the user's diverse needs. For illustration, we provide an example of the parameter calculation process.

Example 4. *Consider a real-world AOL dataset, to be introduced in Section 7.1, with cardinality* $u = 10,000,000$. *Suppose the user expects the FPR to be lower than* $\epsilon = 0.1$ *and no FNs for* $w \leq W_0 = 10,000,000$. *We first calculate the constraint on fingerprint length* f *based on Equation (20),* i.e., $\lceil \log_2 \frac{2\cdot 4}{0.1^2}(2 \cdot 10^{-7} \cdot 10^7 + 0.1 + 2\sqrt{(1+0.1)}) \rceil = \lceil 11.71 \rceil = 12$, *where* $\bar{b} = 4$ *and* $\hat{p}$ *is replaced by* $\frac{1}{u}$ *according to Remark 2. Then, we find the optimal values of* κ *according to Theorem 8,* i.e., $\kappa = \sqrt{\frac{2\cdot 4\cdot 10^7}{2^{12}\cdot 10^{-7}}} \approx 441941$ *and thus* $m = \kappa/b = 110485$ *according to Section 6.3.2. However, the CF requires* m *to be a power of* 2, *so we round* m *to the closest power of 2,* i.e., $m = 2^{17}$, *and inversely derive* $\kappa = 524288$. *Last,* $L = \lceil W_0/\kappa \rceil + 1 = 21$. *To sum up,* $f = 12, b = 4, \kappa = 524288, m = 131,072$ *and* $L = 21$.

The query performance for $w > W_0$ is jointly determined by the CFs and the model. If the incoming data does not follow the training data distribution, the FNR and FPR may increase due to inaccurate gap estimation of the model. The space costs would also increase due to less effective redundancy identification. The experiments in Section 7.3.5 validate this notion and show the robustness of LCF_O when slight distribution changes occur. We assume that the underlying distribution of the data stream is relatively stable over time. This is not a very strong assumption. For example, after the oracle learned that shorter words are more frequent in text data, it is unlikely that longer words suddenly become more frequent. Hence, the time and space overheads associated with retraining are affordable. If the distribution undergoes a radical drift, it is necessary to retrain the model on a subset of the new data stream, following the same methodology in Section 4.2.2. The retraining process can be triggered in two ways: passively, when accuracy falls below a user-defined threshold; or actively, by utilizing various drift detection techniques, as surveyed in [28].

6.4 Complexities

According to Algorithm 3, for each newly arrived element, LCF needs to perform at most $\gamma_1 + \gamma_2$ deletions, one insertion, and one call to the model. According to Algorithm 2, to answer a membership query, LCF needs to perform at most L queries and one call to the model. As proved in [18], cuckoo filters have $O(1)$ amortized insertion time and $O(1)$ query and deletion time. Supposing the model prediction takes $O(\mathcal{M})$ time, the time complexities of Update and Query are $O(\mathcal{M} + \gamma_1 + \gamma_2)$ and $O(\mathcal{M} + L)$, respectively. Since all $\mathcal{M}, \gamma_1, \gamma_2$ and L are constants, the time complexities are $O(1)$. The space cost of LCF comes from two aspects, one is the model size, and the other is the size of CFs $f\bar{b}mL$, where $\bar{b}$ is the average b of CFs after Compact. As shown in Section 7.3.2 and Section 7.3.3, deleting redundant information and compressing sparse CFs by taking advantage of data distribution can save a lot of space, which outweighs the overhead from the model.

7 EXPERIMENTS

In this section, we conduct extensive experiments on both real-world and synthetic datasets to show the superiority of LCF over state-of-the-art algorithms. Unless otherwise specified, in the remaining part, LCF specifically refers to LCF_O. All experiments are conducted on a server running 64-bit Ubuntu 20.04 with 2 Intel(R) Xeon(R) Gold 5218 CPUs @ 2.30GHz.

7.1 Experimental Settings

7.1.1 Datasets. To show the effectiveness of LCF under different application scenarios and data distributions, we employ 2 real-world and 2 synthetic datasets that are also used by competitors:

- **AOL** [1] is a real-world dataset of web query logs collected by search engines over 90 days. There are 10 million distinct search queries in a total of 21 million queries (364 MB in size), where each query consists of multiple words, *e.g.,* "tropical rain forest".
- **CAIDA** [2] is a real-world dataset of anonymized passive traffic traces collected at a backbone link of a Tier-1 ISP between Chicago and Seattle in 2016. We use network packets coming from a 10-minute consecutive trace as the data stream. 7 million records are distinct in 300 million records (1.7 GB in size).
- **Gauss** and **Zipf** are two synthetic datasets following the Gauss and Zipf distributions. Each of them has 20 million unique elements, 1 billion elements in total (7.92 GB and 5.75 GB in size).

7.1.2 Competitors. We compare LCF with three methods as mentioned in Section 2, *i.e.,* Stable BF [15], Sliding BF [19], and Stair BF [47]. Stable BF is a typical filter used in the streaming data scenario. We adopt MurmurHash3[1] as its hash function, which is a widely tested and fast non-cryptographic hash function suitable for general hash-based lookup. To expedite the computation of hash values, we adopt the idea in [22] where only two independent hash values are computed, from which the remaining hash values are derived. Sliding BF and Stair BF are two state-of-the-art filters for AMQs on data streams. Sliding BF is designed to answer AMQs over fixed-size sliding windows. Stair BF can answer whether an element has appeared in a certain period. By checking all periods covered by the sliding window, it can answer AMQs over variable-sized sliding windows, especially when $w \leq W_0$. Both Sliding BF and Stair BF are open-sourced, so we use their original implementations. In addition, to gain insights into how the model and CF components impact the performance and identify areas for improvement, we design two baselines, called only_model and hashmap. As their names imply, the only_model makes non-member decisions based only on the gap of the queried element q outputted by the model, *i.e.,* if $w > \mu_q$, TRUE is returned, otherwise FALSE. The hashmap records the latest arrival timestamps of all elements in the universe. The AMQs are answered as explained in Example 2.

7.1.3 Models. We follow the implementation of the learned oracle in [21]. Specifically, for the CAIDA dataset, we employ Recurrent Neural Networks (RNNs) with 64 hidden units to encode IP addresses and extract the final states as features. Ports are encoded by two-layer fully-connected networks with 16 and 8 hidden units. The encoded IP and port vectors are concatenated with the protocol type and used for frequency estimates via a two-layer fully-connected network with 32 hidden units. We use the first 2-minute data for training and build LCF on subsequent data. To handle the AOL dataset, we train an RNN with 256 hidden units, which accepts characters from a search phrase as input. The RNN's encoded final states are then fed to a fully-connected layer with 32 hidden units to predict frequency. We use the first 4 days for training. For synthetic datasets, we use a two-layer fully-connected network with 200 hidden units and train it on the first 10% of the

[1]https://github.com/aappleby/smhasher/blob/master/src/MurmurHash3.cpp

Table 2. Summary of default parameters

Datasets	b	W_0	ϵ	f	κ	m	L
AOL	4	10,000,000	0.1	12	524,288	131,072	21
CAIDA	4	10,000,000	0.1	12	524,288	131,072	21
Gauss	4	20,000,000	0.1	12	1,048,576	262,144	21
Zipf	4	20,000,000	0.1	12	1,048,576	262,144	21

data stream. The CF component in LCF follows the implementation of the original CF [18]. We have released all the source code of LCF[2].

7.1.4 Paramether Settings. By default, the filters are expected to have a FPR lower than 0.1 and no FNs for $w \leq W_0$, where the default W_0 is 10 million for the AOL and CAIDA datasets and 20 million for the Gauss and Zipf datasets. For LCF, we can calculate the remaining parameters needed on each dataset according to the formulas derived in Section 6. Table 2 summarizes the default parameters of LCF for all datasets. By default, the Compact operation is triggered whenever $\alpha < 0.5\alpha^*$. γ_1 and γ_2 are set to 1 and 2 for real and synthetic datasets, respectively. The parameter settings of competitors follow the original papers. For Sliding BF, we set the number of segments to $k = 15$ and the number of fields in each bucket to $d = 2$. For Stair BF, we set three layers. The sketches in the bottom layer use 4 hash functions while others use 1. For a Stable BF, we set the maximum value of each counter to $MAX = 3$, the number of hash functions to $K = 2$, and the number of random decrements to $P = 15$.

7.1.5 Evaluation Metric. There are four typical metrics in total. Empirical FPR and FNR are used to evaluate query accuracy. Query time is used to evaluate time efficiency, and size is for space efficiency. We insert all elements on data streams into each filter one by one in chronological order and randomly select 20 timestamps to issue membership queries. For a specific timestamp $t_$ and window size $w_$, the elements contained in the window are regarded as positive samples, while others are regarded as negative samples. Let the set of elements within $w_$ at timestamp $t_$ as $\mathbb{U}_p$ and the rest as $\mathbb{U}_n$, then the empirical FPR and FNR over a $w_$ sized sliding window can be calculated as $\frac{\#\text{ of TRUE}}{|\mathbb{U}_n|}$ and $\frac{\#\text{ of FALSE}}{|\mathbb{U}_p|}$, respectively. We use the average value of empirical FPR/FNR at 20 timestamps as the experimental results for $w_$. The query time is measured by # of operations (Queries/Inserts) per second that can effectively reduce the impact of randomness.

7.2 Performance Analysis

7.2.1 Towards variable-sized sliding windows. In this set of experiments, we study the empirical FPR and FNR of all algorithms on all datasets given the same space cost in the default setting. Based on the results shown in Figure 5, we make the following observations: (1) LCF offers the best performance in terms of both FPR and FNR across all query window sizes on all datasets, indicating that LCF achieves the best trade-off between accuracy and space cost. In particular, LCF outperforms Sliding BF by 9× in FPR for the smallest query window on the real datasets (0.035 v.s. 0.306 on AOL and 0.010 v.s. 0.093 on CAIDA). For FNR, LCF shows at most 6× and 134× improvements compared to Stair BF (0.015 v.s. 0.09 when $w = 11.5 \times 10^6$) and Stable BF (0.002 v.s. 0.263 when $w = 10.5 \times 10^6$) on AOL. Since the FNR of Stable BF is considerably worse than others, we do not plot it on Zipf, AOL, and CAIDA. Instead, we show its average FNR in the legend. (2) LCF, Stair BF, and Sliding BF have no FNs when $w \leq W_0$, while Stable BF has FNs for any window sizes. The reasons are as follows: a) Stable BF randomly decrements counters to make

[2]https://github.com/learned-index/LCF

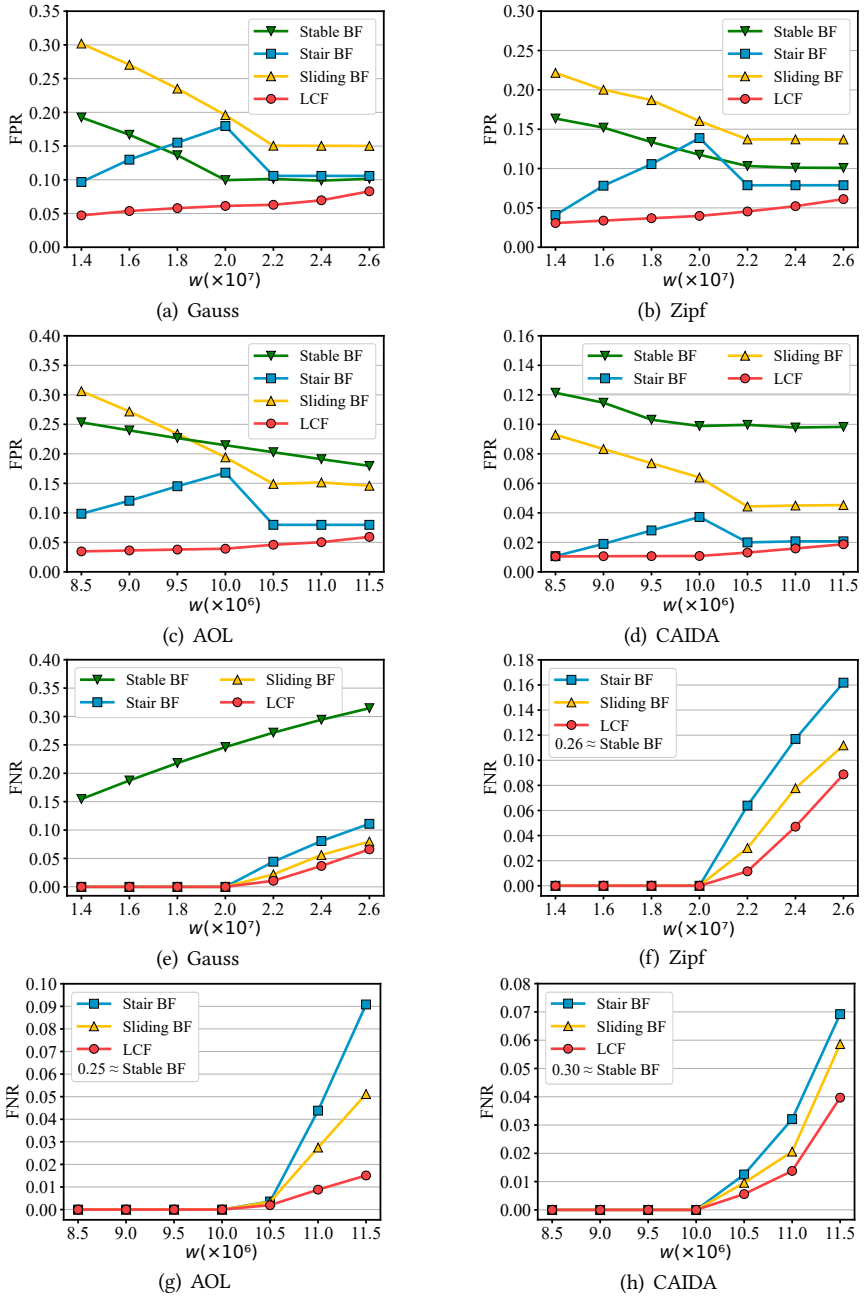


Fig. 5. Empirical FPR/FNR over variable-sized sliding windows

room for fresh elements, which can cause false deletions and prevent Stable BF from having any quality guarantee on FNs; b) neither Stair BF nor Sliding BF has any deletion operations within W_0 elements, and thus there are no FNs; c) LCF can guarantee no false deletions and thus no FNs, which is consistent with Theorem 5. (3) Stable BF and Sliding BF have a similar decreasing trend in the FPR as w increases, while LCF and Stair BF show an increasing trend, *i.e.*, only LCF and Stair BF achieve memorizing recent elements with higher accuracy. The reasons are as follows: a) for the

Table 3. The accuracy with the same space cost

Methods	LCF	Stair BF	Sliding BF	Stable BF	
Metric	FPR	FPR	FPR	FPR	FNR
Gauss	**0.061**	0.180	0.196	0.099	0.246
Zipf	**0.040**	0.139	0.160	0.118	0.270
AOL	**0.039**	0.168	0.194	0.215	0.250
CAIDA	**0.011**	0.037	0.064	0.099	0.303

Table 4. The space cost at the same FPR level

Methods	LCF	Stair BF	Sliding BF	Stable BF	
Metric	Size(MB)	Size(MB)	Size(MB)	Size(MB)	FNR
Gauss	**24**	32	36	24	0.246
Zipf	**19.5**	30	35	18	0.290
AOL	**17**	30	35	10	0.381
CAIDA	**7.8**	15	20	7.8	0.303

window size $w \leq W_0$, neither Stable BF nor Sliding BF can distinguish whether a queried element is in $S_n(w)$ or $S_n(W_0) \backslash S_n(w)$. This causes the negatives in $S_n(W_0) \backslash S_n(w)$ might be wrongly reported as positives, with the probability decreasing as w grows; b) LCF and Stair BF can distinguish finer-grained window sizes by only querying the CFs or BFs covered by the window, where the FPR grows with the number of queries. (4) The FPR of all competitors reaches a constant while their FNR keeps rising with w. The reasons are as follows: a) the fraction of zeros in Stable BF converges to its stable point, so the FPR becomes stable. However, the probability that an element in $S_n(w)$ has been randomly deleted increases with w, resulting in the ever-increasing FNR; b) the information Stair BF and Sliding BF can sketch is the most recently observed W_0 elements, on the basis of which the member/non-member decision is made. As w grows, more elements fall in $S_n(w) \backslash S_n(W_0)$ and can become FN errors. LCF achieves low FPR and FNR at the same time for two reasons: a) for small window sizes $w \leq W_0$, LCF makes a decision on the basis of covered CFs. The intrinsic properties of CF make it possible for LCF to achieve a lower FPR compared to the BF-based methods. The novel Update and Compact operations guarantee that the removals are only redundant information, which enables no FN errors and the same FP errors with less space cost. In other words, LCF has a low FPR at the same space cost; b) for large window sizes $w > W_0$, CFs share a decision with the model. Although the FNR of CF increases for the same reasons as that in Stair BF, the model can grasp the frequency characteristics well to keep the FNR low, to be further discussed in the ablation study.

7.2.2 Towards fixed-size sliding windows. As mentioned earlier, fixed-size sliding windows can be considered a special case of variable-sized sliding windows. We do not expect the proposed filter to sacrifice its performance in the original application scenario in exchange for versatility. Thus, in this set of experiments, we evaluate the query performance over W_0-size sliding windows. Experimental results in the default settings demonstrate that LCF achieves higher accuracy at a smaller space cost than filters dedicated to W_0-size windows, as shown in Table 3 and Table 4. To control the variables in the experiments, we ensure that each filter has the same space cost as LCF on each dataset and compare their FPR and FNR in Table 3. Specifically, M = 24MB on Gauss, 19.5MB on Zipf, 17MB on AOL and 7.8MB on CAIDA. Since LCF, Stair BF, and Sliding BF have no FNs over W_0-size sliding windows, we omit the FNR column. LCF achieves an average

improvement of 4.3× in the FPR. The space required for each method to achieve a near FPR when the window size is W_0 is shown in Table 4. Note that although Stable BF can memorize recent elements by evicting the stale elements, there is no clear definition of the sliding window. For fairness, we adjust its coverage rate through m such that it reaches the default FPR stable point (0.1) at W_0. Adjusting m, and thus the space cost affects only the speed of convergence, not the FPR. We can see that LCF takes around 42% less space than the state-of-the-art fixed-size sliding window filters. In particular, on CAIDA, LCF uses 61% less space than Sliding BF. The recovery of the over-allocated space outweighs the space overhead of the model (0.079MB on Gauss and Zipf, 0.8MB on CAIDA, and 3.9MB on AOL). Although Stable BF takes less space than other filters, its FNR is considerably high. A 30% FNR is unacceptable in many applications. In sum, LCF achieves the best space/accuracy trade-off.

7.3 Further Analysis

7.3.1 Stability. The accuracy also depends on the timing of the query. In this set of experiments, we evaluate the stability of LCF. Due to space limitations, we only report the results on AOL. We randomly pick 100 query times for each window size, *i.e.,* 10 per 2 million inserted elements. The window sizes are $w = 6\times10^6, 8\times10^6$, and 12×10^6, where $12\times10^6 > W_0$ as default. The experimental results are shown in Figure 6. We can observe that when the number of elements in the stream seen so far is smaller than the window size, the FPR gradually increases since more CFs need to be checked. After more data arrive, the FPR for queries with $w < W_0$ stabilizes within a specific range (the blue and yellow boxes), and there are no FNs. When the number of inserted elements exceeds W_0, FNs begin to appear for queries with a large window $w = 12 \times 10^6$, which is also stable.

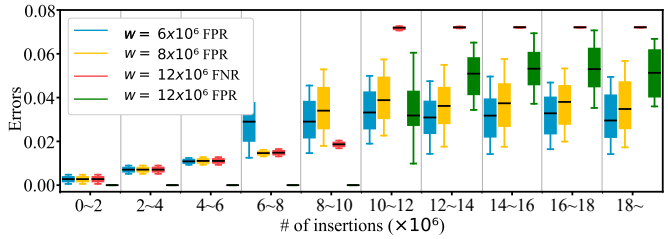


Fig. 6. The stability of LCF

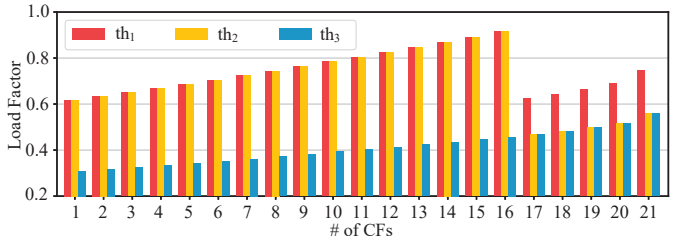


Fig. 7. Effect of Compact

7.3.2 Effect of Update. To verify the effectiveness of the delete operation during Update, we investigate how the load factor in a stale CF changes as the number of Update increases after this CF is full, as shown in Figure 8. For clarity, we do not consider Compact and dynamic removal of expired CFs here. Due to space limitations, we only report the results on Gauss. From the figure, we can observe that the load factor does not increase because newly arrived elements are only inserted into a new CF. Whenever an element is inserted, its old version might be deleted, which gradually decreases the load factor from 92% to a stable point of 40%. The remaining elements either occur

Table 5. LCF_C v.s. LCF_O by varying f and κ

	$f = 8$		$f = 12$		$f = 16$	
Metric	FPR	Size(MB)	FPR	Size(MB)	FPR	Size(MB)
$\kappa = 2^{18}$	0.793	12.4	0.113	23.6	0.009	34.5
$\kappa = 2^{20}$	0.363	15.5	**0.061**	**24**	0.040	35
$\kappa = 2^{22}$	0.243	18	0.159	30	0.144	40

infrequently or recur but are not deleted. About half of the elements are successfully deleted, and thus the space released by them can be reclaimed by Compact to sketch fresh ones. Furthermore, we report specific space costs of LCF_C and LCF_B under different parameter settings to highlight the importance of effective delete operations. When (f, κ) are $(8, 2^{18})$, $(12, 2^{20})$ and $(16, 2^{22})$, LCF_B uses 19.5MB, 31.5MB, and 48MB, respectively. The space costs of LCF_C are summarized in Table 5, which takes 60% - 80% of the space to store the same number of elements.

7.3.3 Effect of Compact. To verify the effectiveness of Compact, we plot the load factors of 21 CFs under different Compact trigger thresholds. Figure 7 presents the results on Zipf under three thresholds, denoted as $th_1 : \alpha < 0.75\alpha^*$, $th_2 : \alpha < 0.5\alpha^*$, and th_3 : not compacted. From the figure, we find that space utilization is only around 40% if not compacted. Under th_2, 1-16 CFs are successfully compressed and their load factors climb back up to a high level. Under th_1, 1-16 CFs are compressed twice, and 17-21 CFs are compressed once, which further eases the space cost. These experiments also offer evidence for the analysis in Section 6.3.2. Despite the possibility of initially over-allocating the space of the CF, LCF can efficiently recover the excess space using Compact.

7.3.4 Effect of parameters. In Table 5, we study the effect of two important parameters, *i.e.,* κ and f, on Gauss while keeping b, W_0 and ϵ at their defaults. It can be observed that LCF_O (bold) can satisfy the specified accuracy with minimal space costs, while un-optimized settings can produce arbitrarily bad results. Increasing the length of the fingerprint can further improve the FPR, which is consistent with theoretical results. To investigate how the hyper-parameters γ_1 and γ_2 balance update efficiency and space consumption, we vary their values in {0, 1, 2, 3}. On the Gauss dataset, we observe that the space costs of LCF are 26.25MB, 25.5MB, 24MB and 23.25MB, when $\gamma = 0, 1, 2$ and 3, respectively. The corresponding time costs can be easily estimated by the number of standard CF deletion/query/insert operations, which are 1, 3, 5, and 7, respectively. We define β as the percentage of successful removal of one or multiple instances of redundancy in old CFs when Updating elements in the stream that trigger the deletion operations. We find $\beta = 57.06\%, 62.72\%, 65.58\%$ and 67.25% when $\gamma = 0, 1, 2$ and 3. As increasing the values of γ_1 and γ_2 does not yield much improvement in the removal rate, we use 1 or 2 as reasonable defaults.

7.3.5 Robustness. These experiments explore the impact of deviations between the incoming data and the training data distribution. To simulate this effect, we change the standard deviation (std) of the Gauss dataset. Figure 10 shows the results under two levels of distribution distortion: *Distortion+*, where the incoming data's std is 5 million, while the model was trained on data with a std of 4 million, and *Distortion++*, which involves greater distortion by training on data with a std of 3 million. For $w \leq W_0$, which depends entirely on CFs, deviations between incoming and training data have no effect (and thus we omit the results for $w < W_0$), whereas error rates increase for $w > W_0$. The space costs for the *Distortion+* and *Distortion++* increase by 12% and 21%, respectively. Overall, the degradation in performance is gradual.

7.3.6 Ablation Study. To verify the significance of combining a traditional data structure and machine learning techniques for quality guarantee, we compare LCF and only_model on Gauss as

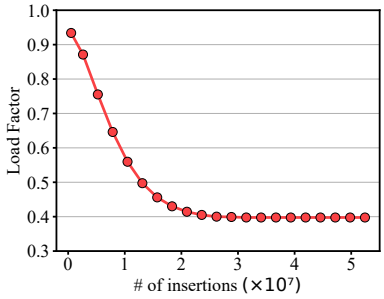


Fig. 8. Effect of UPDATE

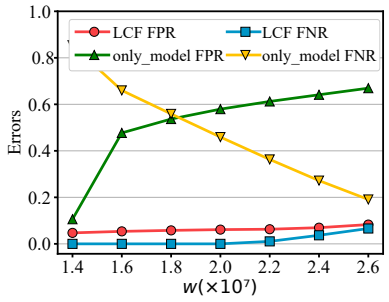


Fig. 9. Effect of model

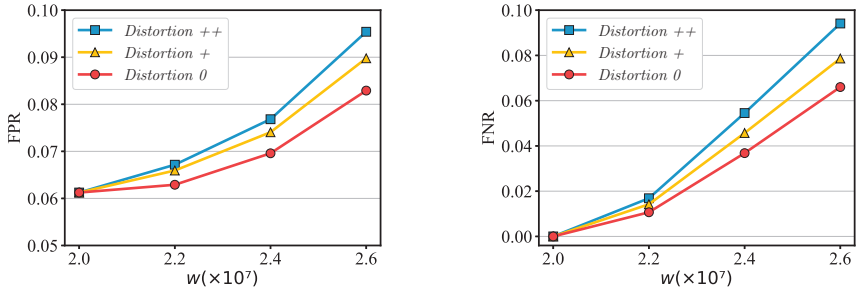


Fig. 10. Effect of different levels of distribution distortion

shown in Figure 9. We find that relying solely on a model that only understands the distribution of gaps is unreliable. When the window size w is small, there are both FNs and FPs because an element e is a positive or negative not only related to its gap but also its last arrival time t, current time n, and w, *i.e.*, even though $\mu_e > w$, we cannot derive $t < n - w$. When w is large, the model tends to always return TRUE, and thus the FPR inevitably decays to 1. This is because the gaps of most elements are smaller than w. Such an unguaranteed performance is not acceptable in real-world applications. By exploiting the good FNR performance of the model at large window sizes and the good FPR performance of CF, LCF achieves both low FPR and FNR. For the hashmap, it can quickly and accurately handle membership queries with any window size. It can process 26 million queries per second, surpassing any filters in this paper at least tenfold. However, the space cost of the hashmap is huge, such as 1.47GB on Gauss, which can be unbearable.

8 CONCLUSION

This paper presents LCF, a novel data structure that guarantees performance for variable-sized sliding windows on data streams. LCF utilizes a learned oracle to insert new arrivals and delete old versions, resulting in significantly reduced space requirements at the same error level. A learning-driven query processing algorithm is developed to aid in answering AMQs. Theoretical analysis shows that LCF has no FNs in a given sliding window and better recent element memorization accuracy. We explicitly derive the expected FNR and FPR for any window size, along with detailed parameter settings. Performance evaluations show that LCF outperforms all competitors in terms of accuracy and space cost, with comparable response time, in both fixed-size and variable-sized sliding-window application scenarios. We are committed to developing the learned database system and plan to deploy LCF to it in the future.

ACKNOWLEDGMENTS

The research work described in this paper was supported by the Hong Kong Research Grants Council (grant# 16202722 – Just in time).

REFERENCES

[1] AOL 2006. Web Search Query Logs. https://jeffhuang.com/search_query_logs.html

[2] CAIDA 2016. The CAIDA UCSD Anonymized Internet Traces. https://www.caida.org/catalog/datasets/passive_dataset

[3] Paulo Sérgio Almeida, Carlos Baquero, Nuno M. Preguiça, and David Hutchison. 2007. Scalable Bloom Filters. *Inf. Process. Lett.* 101, 6 (2007), 255–261.

[4] Luca Baldassarre, Yen-Huan Li, Jonathan Scarlett, Baran Gozcu, Ilija Bogunovic, and Volkan Cevher. 2016. Learning-Based Compressive Subsampling. *IEEE J. Sel. Top. Signal Process.* 10, 4 (2016), 809–822.

[5] Michael A. Bender, Martin Farach-Colton, Rob Johnson, Russell Kraner, Bradley C. Kuszmaul, Dzejla Medjedovic, Pablo Montes, Pradeep Shetty, Richard P. Spillane, and Erez Zadok. 2012. Don't Thrash: How to Cache Your Hash on Flash. *Proc. VLDB Endow.* 5, 11 (2012), 1627–1637.

[6] Burton H. Bloom. 1970. Space/Time Trade-offs in Hash Coding with Allowable Errors. *Commun. ACM* 13, 7 (1970), 422–426.

[7] Fay Chang, Jeffrey Dean, Sanjay Ghemawat, Wilson C. Hsieh, Deborah A. Wallach, Michael Burrows, Tushar Chandra, Andrew Fikes, and Robert E. Gruber. 2008. Bigtable: A Distributed Storage System for Structured Data. *ACM Trans. Comput. Syst.* 26, 2 (2008), 4:1–4:26.

[8] Francis Chang, Kang Li, and Wu-chang Feng. 2004. Approximate Caches for Packet Classification. In *INFOCOM*. IEEE, 2196–2207.

[9] Moses Charikar, Kevin C. Chen, and Martin Farach-Colton. 2004. Finding frequent items in data streams. *Theor. Comput. Sci.* 312, 1 (2004), 3–15.

[10] Hanhua Chen, Liangyi Liao, Hai Jin, and Jie Wu. 2017. The dynamic cuckoo filter. In *ICNP*. IEEE Computer Society, 1–10.

[11] Graham Cormode and S. Muthukrishnan. 2005. An improved data stream summary: the count-min sketch and its applications. *J. Algorithms* 55, 1 (2005), 58–75.

[12] Zhenwei Dai and Anshumali Shrivastava. 2020. Adaptive Learned Bloom Filter (Ada-BF): Efficient Utilization of the Classifier with Application to Real-Time Information Filtering on the Web. In *NeurIPS*.

[13] Mayur Datar, Aristides Gionis, Piotr Indyk, and Rajeev Motwani. 2002. Maintaining Stream Statistics over Sliding Windows. *SIAM J. Comput.* 31, 6 (2002), 1794–1813.

[14] Niv Dayan, Ioana Bercea, Pedro Reviriego, and Rasmus Pagh. 2023. InfiniFilter: Expanding Filters to Infinity and Beyond. In *SIGMOD Conference*. ACM.

[15] Fan Deng and Davood Rafiei. 2006. Approximately detecting duplicates for streaming data using stable bloom filters. In *SIGMOD Conference*. ACM, 25–36.

[16] Sourav Dutta, Ankur Narang, and Suman K. Bera. 2013. Streaming Quotient Filter: A Near Optimal Approximate Duplicate Detection Approach for Data Streams. *Proc. VLDB Endow.* 6, 8 (2013), 589–600.

[17] Cristian Estan and George Varghese. 2003. New directions in traffic measurement and accounting: Focusing on the elephants, ignoring the mice. *ACM Trans. Comput. Syst.* 21, 3 (2003), 270–313.

[18] Bin Fan, David G. Andersen, Michael Kaminsky, and Michael Mitzenmacher. 2014. Cuckoo Filter: Practically Better Than Bloom. In *CoNEXT*. ACM, 75–88.

[19] Xiangyang Gou, Yinda Zhang, Zhoujing Hu, Long He, Ke Wang, Xilai Liu, Tong Yang, Yi Wang, and Bin Cui. 2022. A Sketch Framework for Approximate Data Stream Processing in Sliding Windows. *IEEE Trans. Knowl. Data Eng.* (2022).

[20] Deke Guo, Jie Wu, Honghui Chen, Ye Yuan, and Xueshan Luo. 2010. The Dynamic Bloom Filters. *IEEE Trans. Knowl. Data Eng.* 22, 1 (2010), 120–133.

[21] Chen-Yu Hsu, Piotr Indyk, Dina Katabi, and Ali Vakilian. 2019. Learning-Based Frequency Estimation Algorithms. In *ICLR (Poster)*. OpenReview.net.

[22] Adam Kirsch and Michael Mitzenmacher. 2008. Less hashing, same performance: Building a better Bloom filter. *Random Struct. Algorithms* 33, 2 (2008), 187–218.

[23] Eric R. Knorr, Baptiste Lemaire, Andrew Lim, Siqiang Luo, Huanchen Zhang, Stratos Idreos, and Michael Mitzenmacher. 2022. Proteus: A Self-Designing Range Filter. In *SIGMOD Conference*. ACM, 1670–1684.

[24] Georgia Koloniari, Nikos Ntarmos, Evaggelia Pitoura, and Dimitris Souravlias. 2011. One is enough: distributed filtering for duplicate elimination. In *CIKM*. ACM, 433–442.

[25] Tim Kraska, Alex Beutel, Ed H. Chi, Jeffrey Dean, and Neoklis Polyzotis. 2018. The Case for Learned Index Structures. In *SIGMOD Conference*. ACM, 489–504.

[26] Shangsen Li, Lailong Luo, Deke Guo, and Yawei Zhao. 2021. Stable Cuckoo Filter for Data Streams. In *ICPADS*. IEEE, 138–145.

[27] Qiyu Liu, Libin Zheng, Yanyan Shen, and Lei Chen. 2020. Stable Learned Bloom Filters for Data Streams. *Proc. VLDB Endow.* 13, 11 (2020), 2355–2367.

[28] Jie Lu, Anjin Liu, Fan Dong, Feng Gu, João Gama, and Guangquan Zhang. 2019. Learning under Concept Drift: A Review. *IEEE Trans. Knowl. Data Eng.* 31, 12 (2019), 2346–2363.

[29] Siqiang Luo, Subarna Chatterjee, Rafael Ketsetsidis, Niv Dayan, Wilson Qin, and Stratos Idreos. 2020. Rosetta: A Robust Space-Time Optimized Range Filter for Key-Value Stores. In *SIGMOD Conference*. ACM, 2071–2086.
[30] Saugat Majumdar, Dhananjay Kulkarni, and Chinya V. Ravishankar. 2007. Addressing Click Fraud in Content Delivery Systems. In *INFOCOM*. IEEE, 240–248.
[31] Ahmed Metwally, Divyakant Agrawal, and Amr El Abbadi. 2005. Duplicate detection in click streams. In *WWW*. ACM, 12–21.
[32] Michael Mitzenmacher. 2018. A Model for Learned Bloom Filters and Optimizing by Sandwiching. In *NeurIPS*. 462–471.
[33] Ali Mousavi, Ankit B. Patel, and Richard G. Baraniuk. 2015. A deep learning approach to structured signal recovery. In *Allerton*. IEEE, 1336–1343.
[34] Moni Naor and Eylon Yogev. 2015. Tight Bounds for Sliding Bloom Filters. *Algorithmica* 73, 4 (2015), 652–672.
[35] Rasmus Pagh, Gil Segev, and Udi Wieder. 2013. How to Approximate a Set without Knowing Its Size in Advance. In *FOCS*. IEEE Computer Society, 80–89.
[36] Yanqing Peng, Jinwei Guo, Feifei Li, Weining Qian, and Aoying Zhou. 2018. Persistent Bloom Filter: Membership Testing for the Entire History. In *SIGMOD Conference*. ACM, 1037–1052.
[37] David M. W. Powers. 1998. Applications and Explanations of Zipf's Law. In *CoNLL*. ACL, 151–160.
[38] Hong Shen and Yu Zhang. 2008. Improved Approximate Detection of Duplicates for Data Streams Over Sliding Windows. *J. Comput. Sci. Technol.* 23, 6 (2008), 973–987.
[39] Ariel Shtul, Carlos Baquero, and Paulo Sérgio Almeida. 2020. Age-Partitioned Bloom Filters. *CoRR* abs/2001.03147 (2020).
[40] Rajath Subramanyam, Indranil Gupta, Luke M. Leslie, and Wenting Wang. 2016. Idempotent distributed counters using a forgetful bloom filter. *Clust. Comput.* 19, 2 (2016), 879–892.
[41] Kapil Vaidya, Tim Kraska, Subarna Chatterjee, Eric R. Knorr, Michael Mitzenmacher, and Stratos Idreos. 2022. SNARF: A Learning-Enhanced Range Filter. *Proc. VLDB Endow.* 15, 8 (2022), 1632–1644.
[42] Minmei Wang, Mingxun Zhou, Shouqian Shi, and Chen Qian. 2019. Vacuum Filters: More Space-Efficient and Faster Replacement for Bloom and Cuckoo Filters. *Proc. VLDB Endow.* 13, 2 (2019), 197–210.
[43] Yuhan Wu, Jintao He, Shen Yan, Jianyu Wu, Tong Yang, Olivier Ruas, Gong Zhang, and Bin Cui. 2022. Elastic Bloom Filter: Deletable and Expandable Filter Using Elastic Fingerprints. *IEEE Trans. Computers* 71, 4 (2022), 984–991.
[44] MyungKeun Yoon. 2010. Aging Bloom Filter with Two Active Buffers for Dynamic Sets. *IEEE Trans. Knowl. Data Eng.* 22, 1 (2010), 134–138.
[45] Fan Zhang, Hanhua Chen, Hai Jin, and Pedro Reviriego. 2021. The Logarithmic Dynamic Cuckoo Filter. In *ICDE*. IEEE, 948–959.
[46] Huanchen Zhang, Hyeontaek Lim, Viktor Leis, David G. Andersen, Michael Kaminsky, Kimberly Keeton, and Andrew Pavlo. 2018. SuRF: Practical Range Query Filtering with Fast Succinct Tries. In *SIGMOD Conference*. ACM, 323–336.
[47] Yikai Zhao, Yubo Zhang, Pu Yi, Tong Yang, Bin Cui, and Steve Uhlig. 2022. The Stair Sketch: Bringing more Clarity to Memorize Recent Events. In *ICDE*. IEEE, 164–177.

Received April 2023; revised July 2023; accepted August 2023